

Architecting Java

Object-Oriented Programming I
Master Revision Deck

Derived from SOEN 240 /
Dr.Okemwa (Kisii University)

Syntax

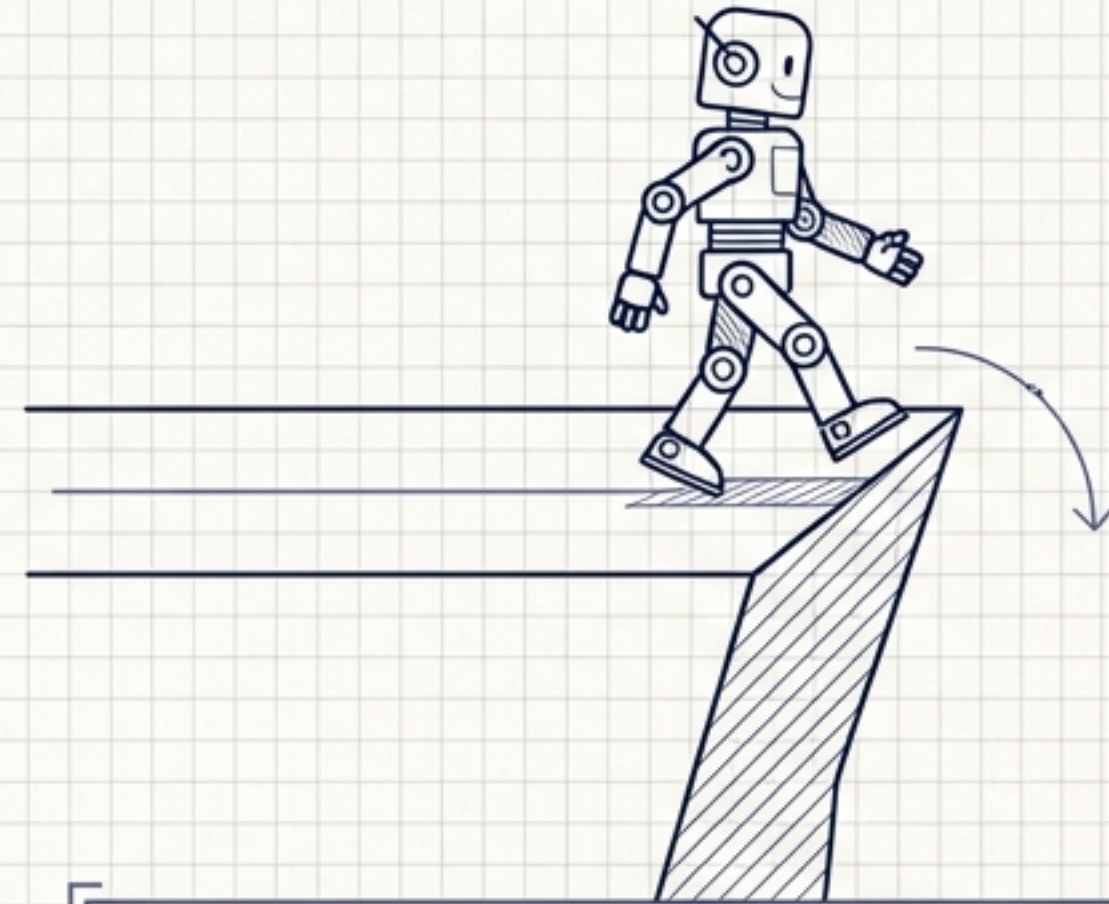
The rules determining how symbols, reserved words, and identifiers form a valid statement.

Colorless green ideas
sleep furiously

```
int age = "hello";
```

Semantics

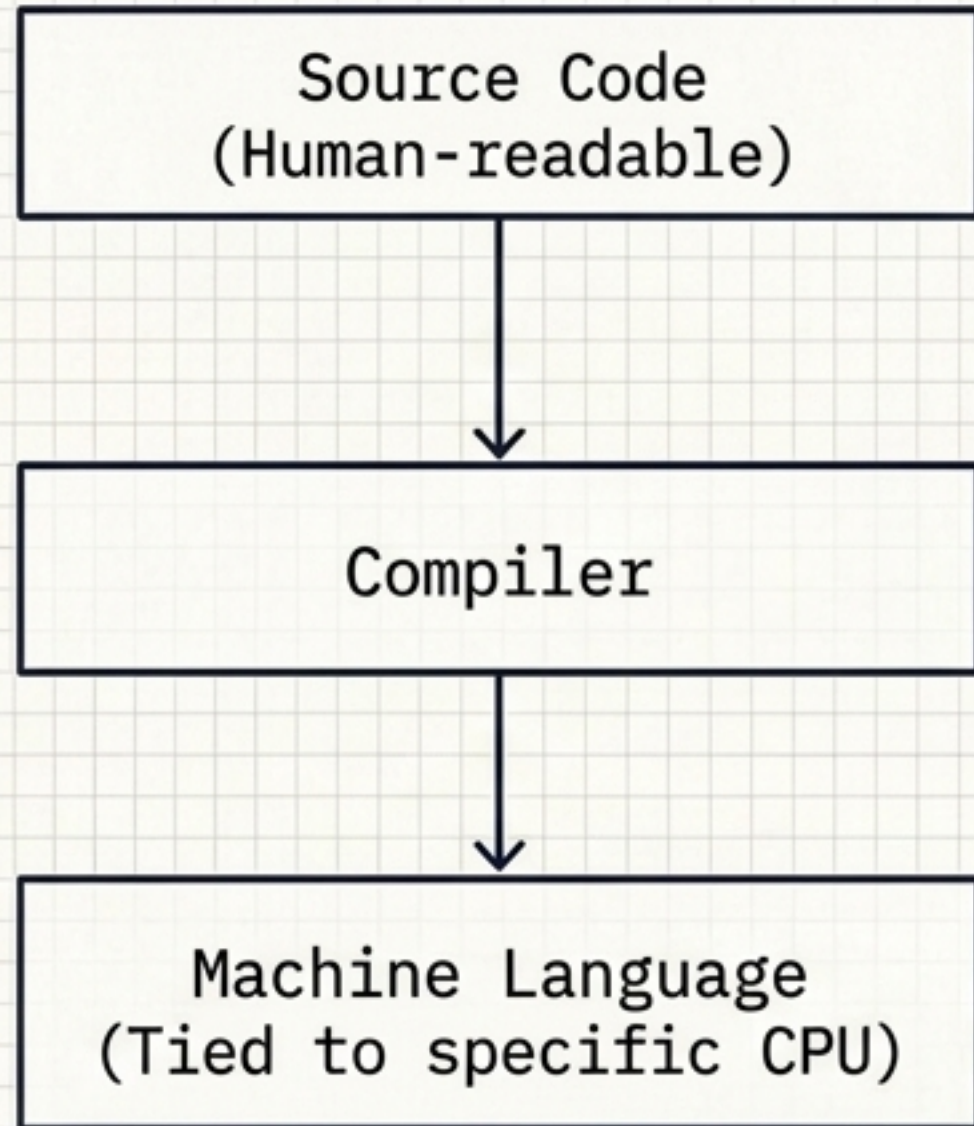
The **actual purpose** and **logic** of a program statement.



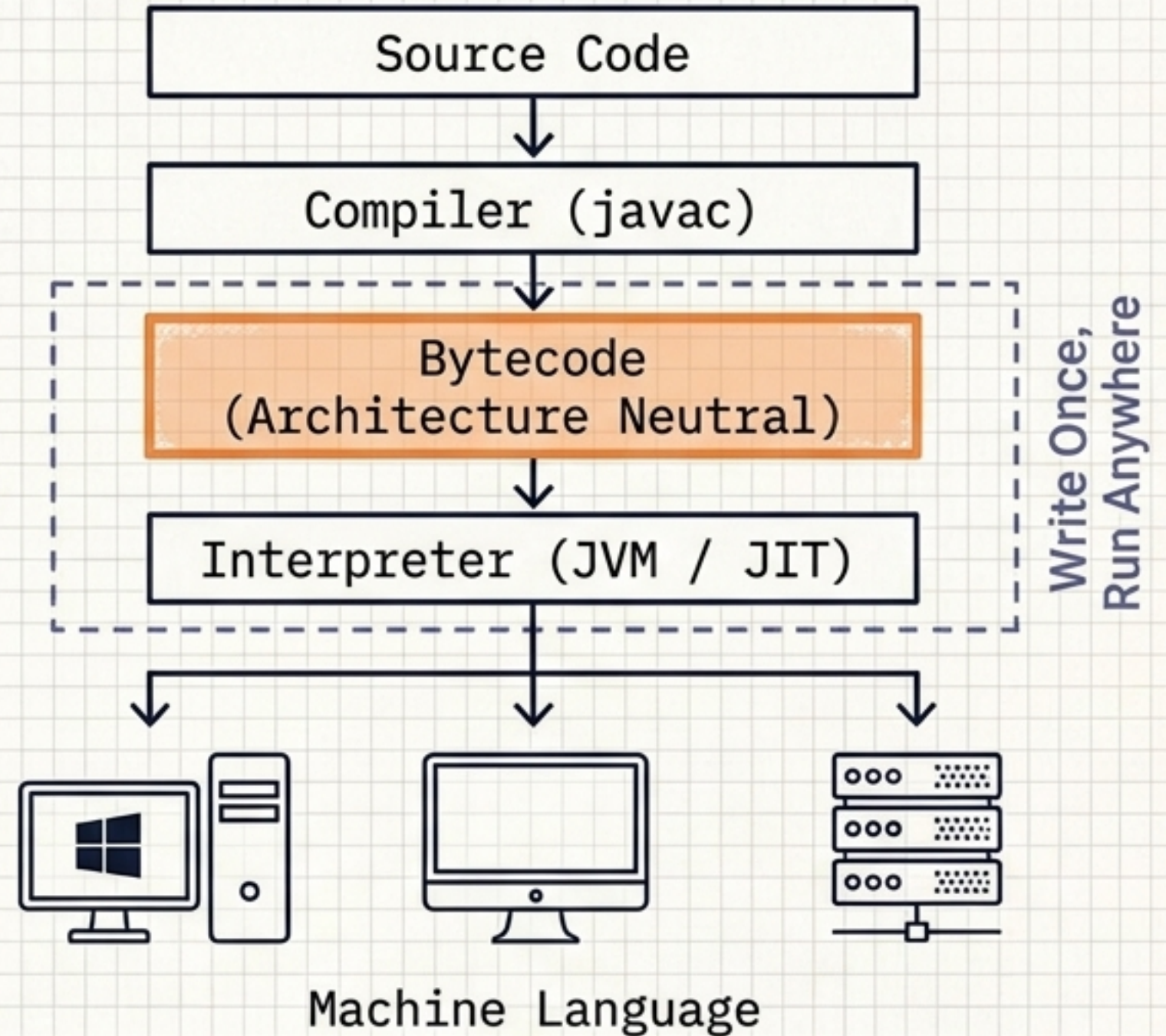
A program does what you tell it to
to do, not what you meant.

The Translation Pipeline

Standard Compiler



Java's Hybrid Architecture



Created by James Gosling (Sun Microsystems, 1995)



Simple & Familiar

C/C++ style syntax.



Object-Oriented

Everything built around objects/classes.



Robust & Secure

Sandbox, bytecode verification.



Portable & Neutral

JVM execution across platforms.



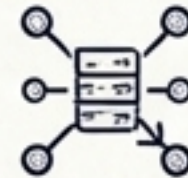
High Performance

JIT (Just-In-Time) compilation.



Multithreaded

Concurrent programming.



Dynamic & Distributed

Runtime loading, networked.



Garbage Collected

Automatic memory reclamation.

JDK (Java Development Kit)

The Core Engine

The essential under-the-hood tools required to build and run Java.

- Compiler
- JVM
- Core libraries

Integrated Development Environments

The Workshop



NetBeans



Eclipse



IntelliJ IDEA



Android Studio

Command Line

The Manual Override

```
> javac Program.java  
> java Program
```

Compile first, then execute.

Package Declaration:
Optional grouping to avoid
naming conflicts.

Import Statements:
Pulls in external libraries.

Comments:
Ignored by compiler.
Used for notes and
javadocs.

```
package myApp;  
import java.util.*;  
// single /* multi */ /** doc */  
public class MyClass  
{  
    public static void main(String[] args)  
    {  
    }  
}
```

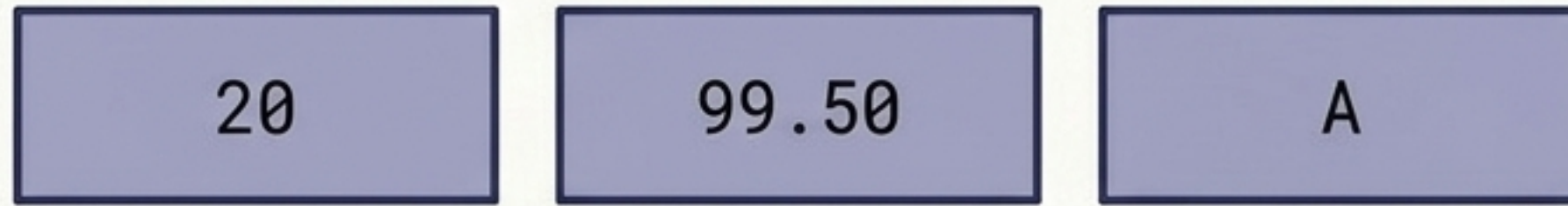
Class Definition:
The outer structural
boundary (Blueprint).

The Main Method:
The required entry point
for execution.

Scope Brackets:
Defines where blocks of
code begin and end.

Primitive Types (Built-In)

Stores the actual value directly in memory.



Integer (byte, short, int, long),
Float (float, double),
Character (char),
Boolean (boolean).

Reference Types

Reference Types

Stores a memory address (reference) pointing to a complex object elsewhere.



Objects,
Arrays,
Strings.

Modifier

(Who can access this method).

Return Type

(What data it gives back. Use 'void' if nothing).

Name

(The identifier used to call the method).

```
public int calculateTotal(int price) { ... }
```

Parameters

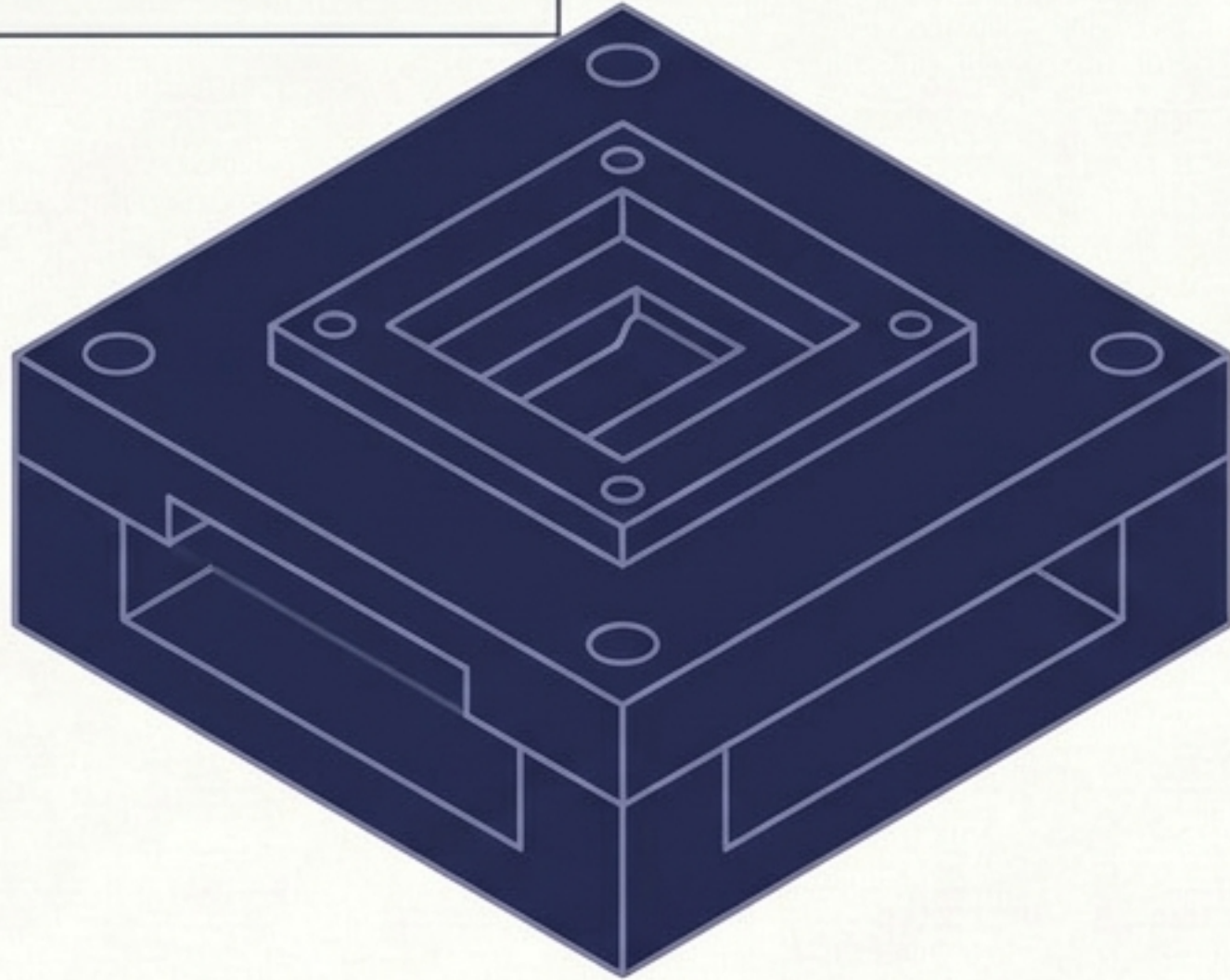
(Arguments passed in to be used).

Body (Contains logic and local variables that exist only while executing).

The Main Method Rule

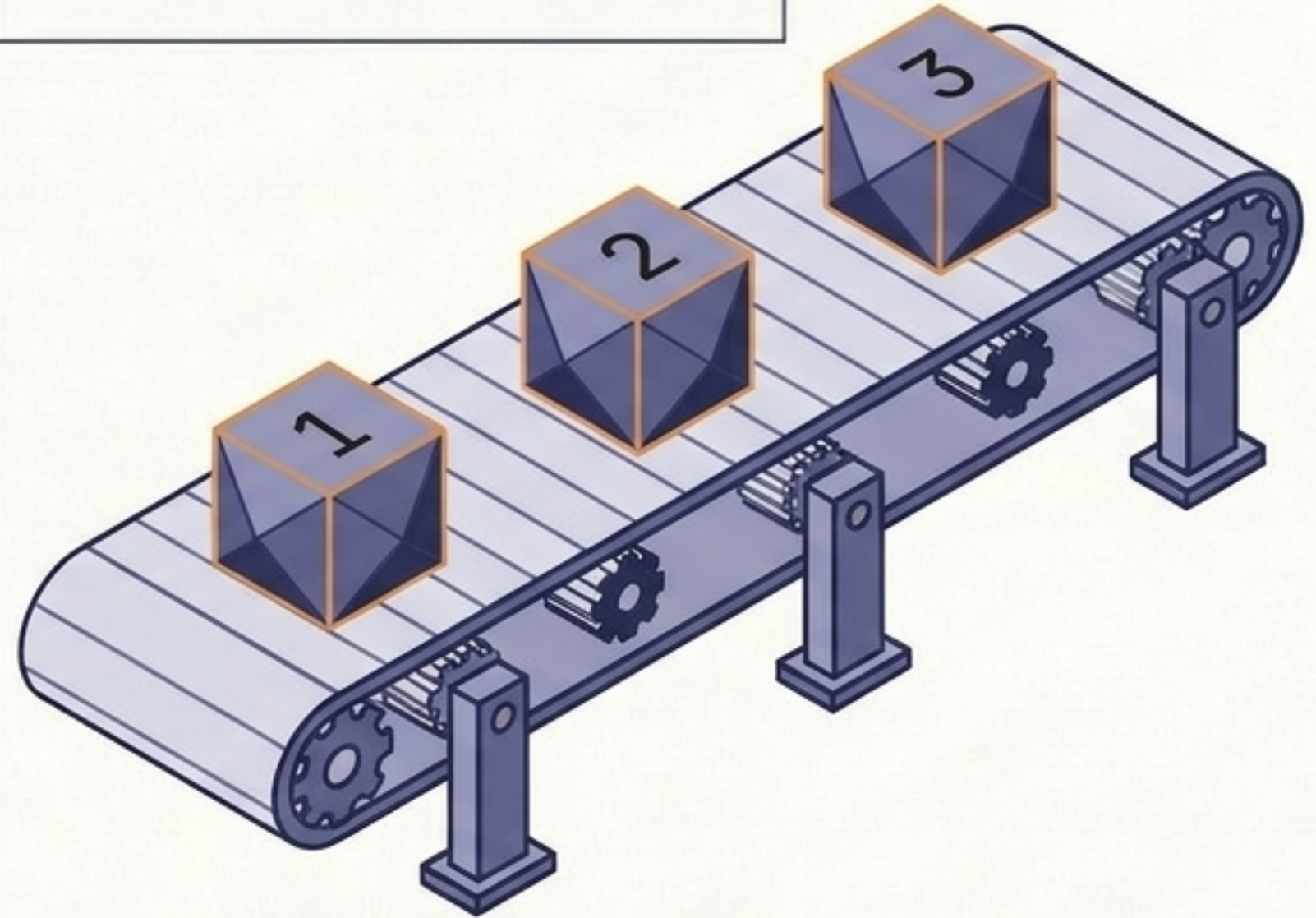
`public static void main(String[] args)` is the ultimate starting point for any Java application. Execution begins here.

CLASS



A blueprint or template detailing attributes (data) and behavior (methods). It does not hold physical data itself.

OBJECTS



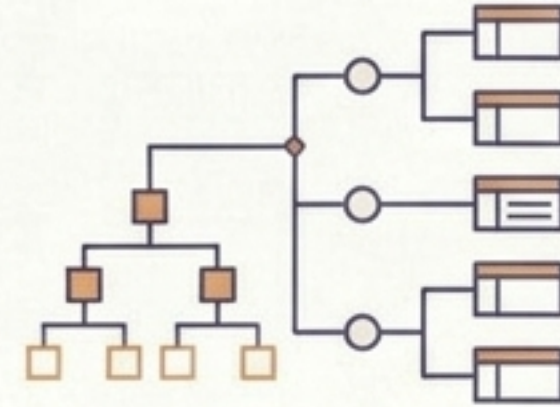
Concrete realizations (instances) of the class blueprint. Created in memory when the program runs.

	Same Class	Same Package	Subclass	Global
public Accessible from anywhere.				
protected Accessible inside same package and subclasses.				
default (no keyword) Accessible only inside the same package.				
private Accessible only inside the same class.				



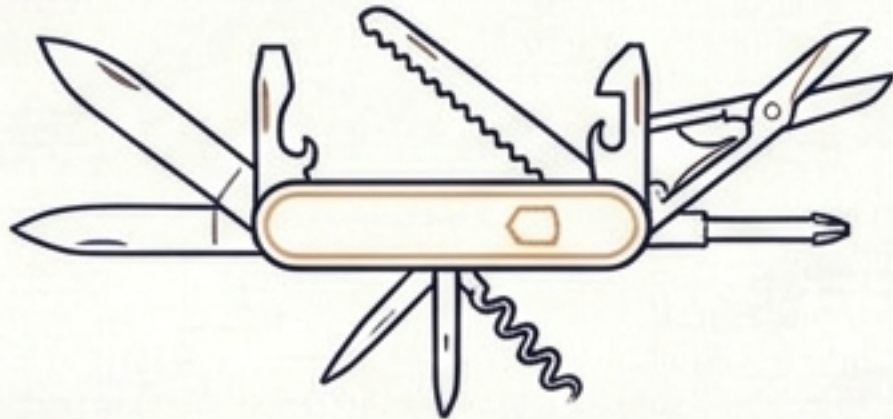
Encapsulation

Bundling data and methods that operate on that data together, controlling access to the internals.



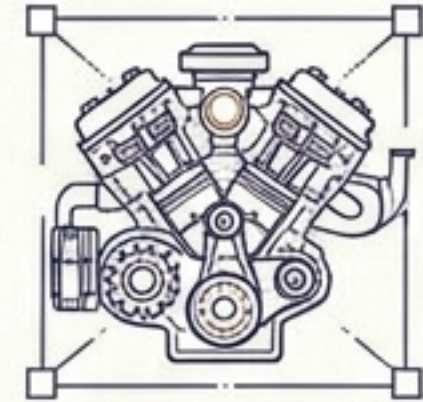
Inheritance

Allowing a new class to inherit attributes and methods from an existing one, promoting reuse.



Polymorphism

The ability of different objects to respond to the exact same message/method in their own distinct ways.



Abstraction

Hiding complex implementation details and exposing only the necessary features to the user.

package

Groups related classes, avoids naming conflicts, manages large projects.

class

The structural blueprint containing variables and methods.

Reserved Words

Keywords with special meaning locked by the compiler. Cannot be used as variable or class names.

class	public	static	
void	int	if	
else	while	for	
return	new	this	super

Quick Revision Checklist

- ✓ Java is compiled to bytecode, then executed by the JVM.
- ✓ Architecture-neutral and portable (“**w**rite once, run anywhere”).
- ✓ **Syntax** = structure rules; **Semantics** = logic/meaning.
- ✓ **C**lass = blueprint; **O**bject = concrete instance.
- ✓ 4 Pillars: Encapsulation, Inheritance, Polymorphism, Abstraction.
- ✓ Access modifiers scale security: **public**, **protected**, **default**, **private**.
- ✓ Every application requires a **main** method entry point.
- ✓ **P**rimitives store actual values; **R**eferences store memory addresses.

Don't just write code. Architect systems.

Mastering Java means understanding the space where your code lives—from the JVM pipeline down to the memory addresses.